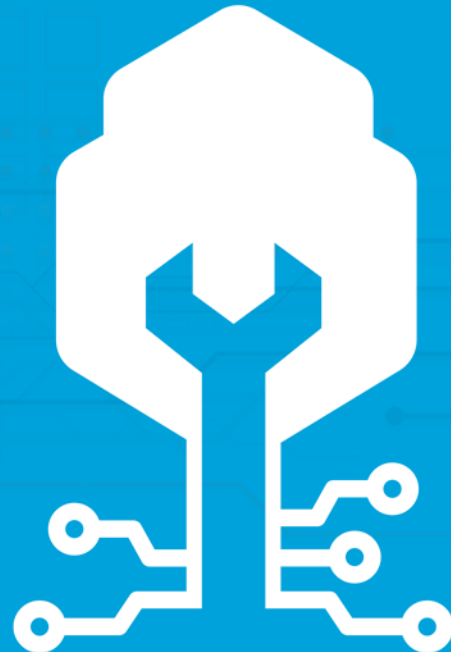


Databases

KEN2110

Denise Slenter

Adapted with gratitude from the original lectures by Dr. Ashish Sai,
Dr. Katharina Schneider, Dr. Max Sondag, and dr. Tom Pepels



Outline for this class



What are we doing today?

- Course details
 - Group project
- ILOs of this course
- Introduction to databases
 - Example:
 - What do we want to store:
 - How do we want to structure our data: entities, relationships, attributes
 - How do we want to store our data: file-based system?
 - Basic knowledge about database systems

Course details



Some of the basics (more details in Syllabus):

- Description: This course will cover data modelling, the concepts and theory of the relational data model, and the widely used programming language SQL. These concepts will be applied in case studies.
- Prerequisites: Procedural Programming and Objects in Programming.
- Grade:
 - 75% exam (minimum grade of 5.5 needed to pass this course!)
 - Closed book exam (TestVision)
 - 25% group project
 - groups of 3 to 4 students

Group Project

- In groups of 3 to 4 students, structure knowledge from a societal challenge into a database.
- Groups open on Monday 31st of August at 13.00 (after this lecture), close on Tuesday 1st of September 18.00 h CEST.
- The project Timeline (Canvas) gives you a structured overview of tasks, deliverables, and opportunities to receive and provide feedback.
 - Assignment 1: Societal Problem Definition
 - Assignment 2: Data Modeling
 - Assignment 3: Schema Design & Database Connection in Code
 - Assignment 4: Video presentation of product to customers
 - Assignment 5: Integration of real data
 - Assignment 6: project wrap up

Group Project

- The project Timeline (Canvas) gives you a structured overview of tasks, deliverables, and opportunities to receive and provide feedback.
 - Assignment 1: Societal Problem Definition
 - Peer review
 - Assignment 2: Data Modeling
 - Peer review
 - Assignment 3: Schema Design & Database Connection in Code
 - Feedback by TAs
 - Evaluate group performance
 - Assignment 4: Video presentation of product to customers
 - Peer review
 - Assignment 5: Integration of real data
 - Peer review
 - Assignment 6: project wrap up
 - Final grade by course coordinator

Literature

- Required:
 - A. Beaulieu, Learning SQL, 2020 (3rd edition, O'Reilly).
- Recommended:
 - M. Kleppmann, Designing Data-Intensive Applications, 2017 (1st edition), O'Reilly



ILOs



Learning Objectives

By the end of this course, you should be able to:

ILO 1: Knowledge and understanding:

- Describe the basic concepts of databases
- Explain the fundamental concepts of database management systems and query languages



Learning Objectives

By the end of this course, you should be able to:

ILO 2: Applying knowledge and understanding:

- Explain proper database design based on the users' requirements
- Indicate possibilities and limitations of each database type
- Combine fundamental Computer Science architectures
- Cooperate in a group to design and construct a database



Learning Objectives

By the end of this course, you should be able to:

ILO 3: Making judgement:

- Analyze and justify a practical database problem
- Examine different design models
- Refine database models based on real-life use cases



Learning Objectives

By the end of this course, you should be able to:

ILO 4: Communication:

- Summarize the basic entities and relationships involved in the database design
- Communicate with the end users to fetch the database requirements



Learning Objectives

By the end of this course, you should be able to:

ILO 5: Learning skills:

- Reflect on their own understanding and identify gaps in their knowledge of database concepts and implementation.
- Take initiative to address these gaps by consulting literature, experimenting with designs, or asking targeted questions.
- Evaluate the correctness and completeness of database models.
- Develop the skills necessary to continue learning about database systems independently, beyond the scope of the course.

Topics covered during this course (lectures, tutorials, group work)

Main topics for SQL:

1. Creating Tables, Selecting data, Updating database, Inserting and Deleting data
2. Defining an ERD
3. Filtering, Querying several tables (joins), sets
4. Grouping and Aggregating, Subqueries
5. Transactions, Indexes and constraints, Views

We will also cover noSQL:

1. Graph databases (e.g. Neo4j), Cypher query language

Group work (and future research projects):

1. Defining a problem statement
2. Working with Big Data on Data Science topics



Introduction to Databases



Why Do We Need Databases?

The Quest for Optimized Data Management

Scenario: A large online bookstore

- **Volume:** Millions of books, users, and transactions → not scalable with text files or memory alone
- **Search:** Need fast lookups beyond $O(N)$ scans
- **Integrity:** Updates must be consistent, no duplicate or stale data
- **Relationships:** Link users, books, and transactions efficiently
- **Concurrency:** Many operations at once without conflicts
- **Recovery:** Survive crashes and restore lost data

👉 Databases exist to solve exactly these challenges.

Without a Database

Store in text files / memory → not scalable

Searching is slow ($O(N)$)

Duplicate / inconsistent data likely

Hard to manage relationships between entities

Concurrency conflicts

No guaranteed recovery after crashes

With a Database

Scales to millions of records

Efficient querying & indexing

Ensures integrity & consistency

Manages relationships with schemas & joins

Supports safe concurrent access

Provides backup & recovery mechanisms

The Basics: What Do We Even Do With Data?

Storing Data:

Like putting your books on a shelf, but way more organized.

Retrieving Data:

Finding that one book in a library of thousands. Preferably quickly.

Updating Data:

That moment when Harry Potter gets a new cover edition for the 1000th time.

Deleting Data:

When you finally decide to Marie Kondo your database.

Searching and Querying:

Looking for all books written by "J.K. Rowling" that are not about wizards.

Relating Data:

If 'Hermione Granger' exists in the Characters table, she better exist in the Wizarding World too!

Imagine: I am the boss of the Limburg Mafia ¹



What's a boss gotta have in his toolbox, eh?

- I want to organise my group of “picciotti”
- To achieve more efficiency in all our operations

Efficient Mafia

- Despite what you might think
- Being Mafia boss requires a lot:
 - Security
 - Privacy
 - Book-keeping (might be risky, but hey, some like control)

I'LL MAKE YOU AN
OFFER YOU
CAN'T REFUSE



The offer...

You've been handpicked by the family, kid.

Your job? To architect our information system. Do us proud.



Efficient Mafia Data: Personnel & Partners

- I want to keep track of
 - People that work for me (Soldiers, caporegime, etc.) 
 - Organizations I do business with (police, 'Ndrangheta, politicians) 
 - Assignments of soldiers to operations
 - Completed and open operations (protection rackets, loan sharking, control of contracting/politics)

Efficient Mafia Data: Operations & Guidelines

- I need to avoid any of my people being involved in:
 - Burglary, mugging, kidnapping (too much police attention) 🚓
- Daily Reporting by Soldiers (6424) 📊
 - Misdeeds in a daily log 📄
 - Money: expenses and collections 💰

Efficient Mafia Data

- Data Sharing with External Organizations, protection some of the information 🤝
 - For business purposes
- I need Accountant Access 📁 (all 20 of the accountants)
 - Money bookkeeping
 - Manage money laundering operations
 - Bank transfers
 - Record bribing expenses
- Semi-Public Interface for other bosses I collaborate with 🤝
 - Search for cops on our books 🚔
 - Check controlled areas 🗺️
 - Plan coups d'état 🔄

Efficient Mafia Data

- Thus, I need Access Control 
 - Boss, Underboss, and Consigliere (advisors) have full access and need to be able to:
 - Assign soldiers to operations.
 - Manage operations (create/shutdown)
 - Financial transactions (e.g., paying cops)
 - Monitor financial flows

Mafia Information System



- Success:  +  +  = Great Life!
- Failure:  Let's not go there...

What does the system need to store?



1 min: write down the entities you expect
1 min: discuss with your neighbor



What does the system need to store



- I want to keep track of
 - **People** that work for me (Soldiers, caporegime, etc.) 
 - **Organizations** I do business with (police, 'Ndrangheta, politicians) 
 - **Assignments** of soldiers to operations
 - Completed and open **operations** (protection rackets, loan sharking, control of contracting/politics)

What does the system need to store

Members

Organizations

Operations

Assignments (who
is doing what)

Payments/
transactions/
money

I need Accountant Access  (all 20 of the accountants)

- Money bookkeeping
- Manage money laundering operations
- Bank transfers
- Record bribing expenses

What does the system need to store



Daily Reporting by Soldiers (6424)

- Misdeeds in a daily log 📄
- Money: expenses and collections 💰

What does the system need to store



- I need to avoid any of my people being involved in:
Burglary, mugging, kidnapping (too much police attention) 🚓

What does the system need to store

Members

Reports

Roles

Organizations

Guidelines

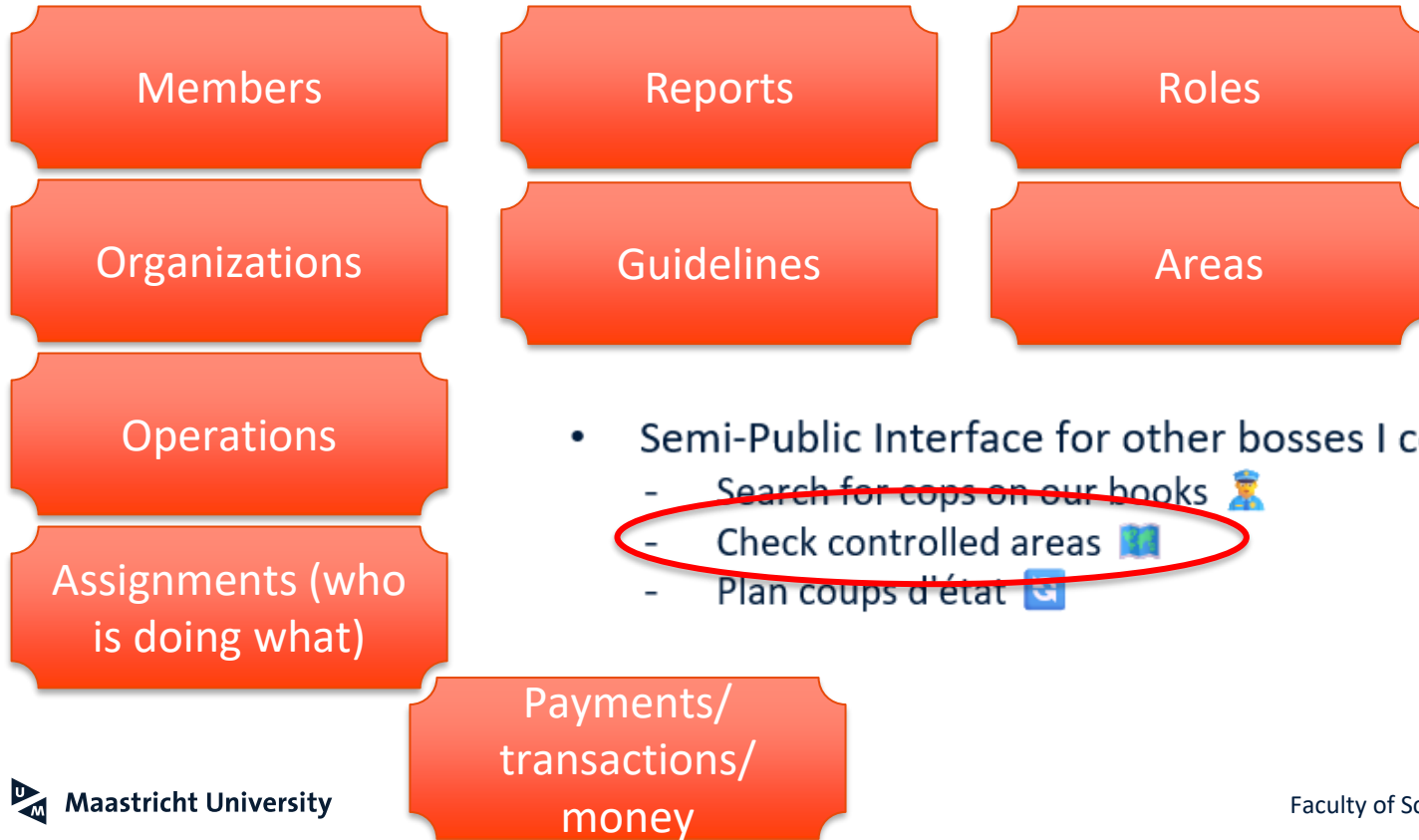
Operations

Assignments (who
is doing what)

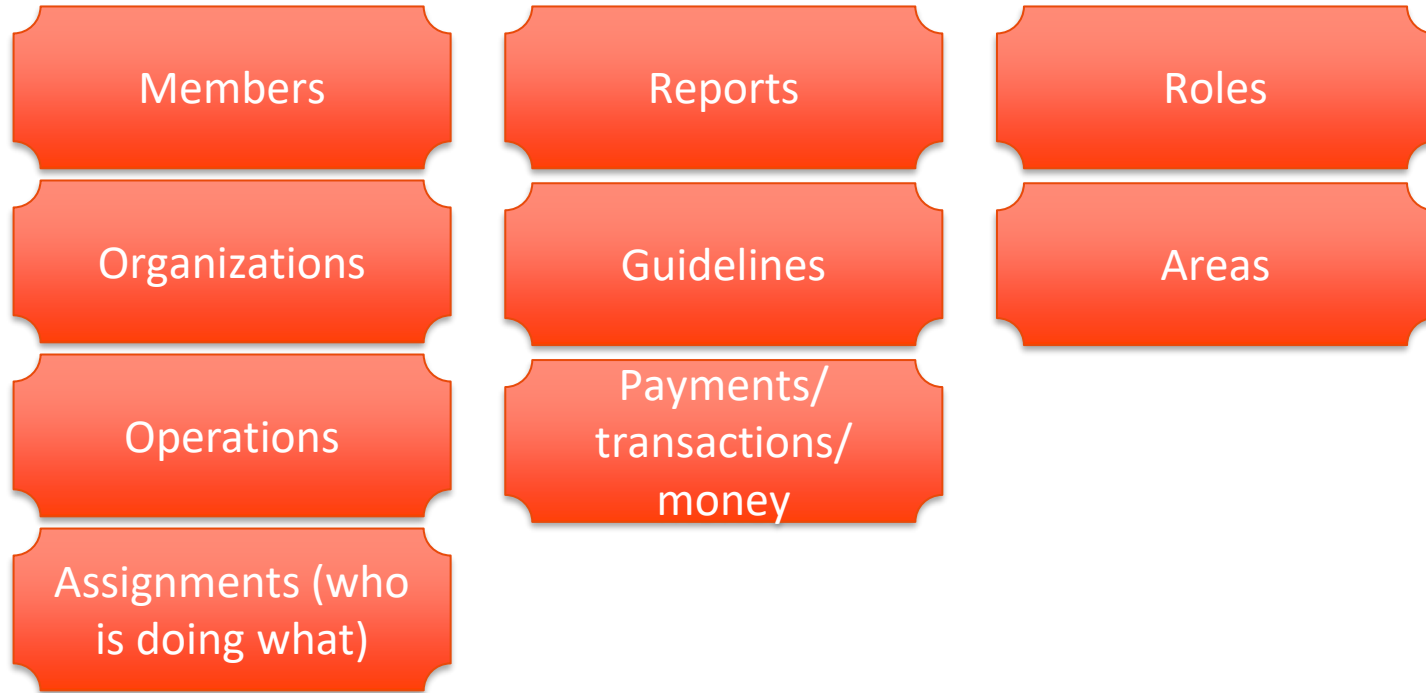
Payment
transaction
money

- Data Sharing with External Organizations, protection some of the information 🧡
 - For business purposes
- I need Accountant Access 🗑️ (all 20 of the accountants)
 - Money bookkeeping
 - Manage money laundering operations
 - Bank transfers
 - Record bribing expenses
- Semi-Public Interface for other bosses I collaborate with 🧡
 - Search for cops on our books 🚔
 - Check controlled areas 📖
 - Plan coups d'état 🗳️

What does the system need to store



What does the system need to store



How do we structure our data?



Entities and Relationships

- Entity: (independent) **object** in the real world that the system needs to store information about
- Relationship: describes how entities are **connected**
- Attributes: **properties** we stored about entities or relationships

Entities and Relationships

- There is not only one correct answer to what an entity is in your system
- It depends on your interpretation and on the context

Independent Entities

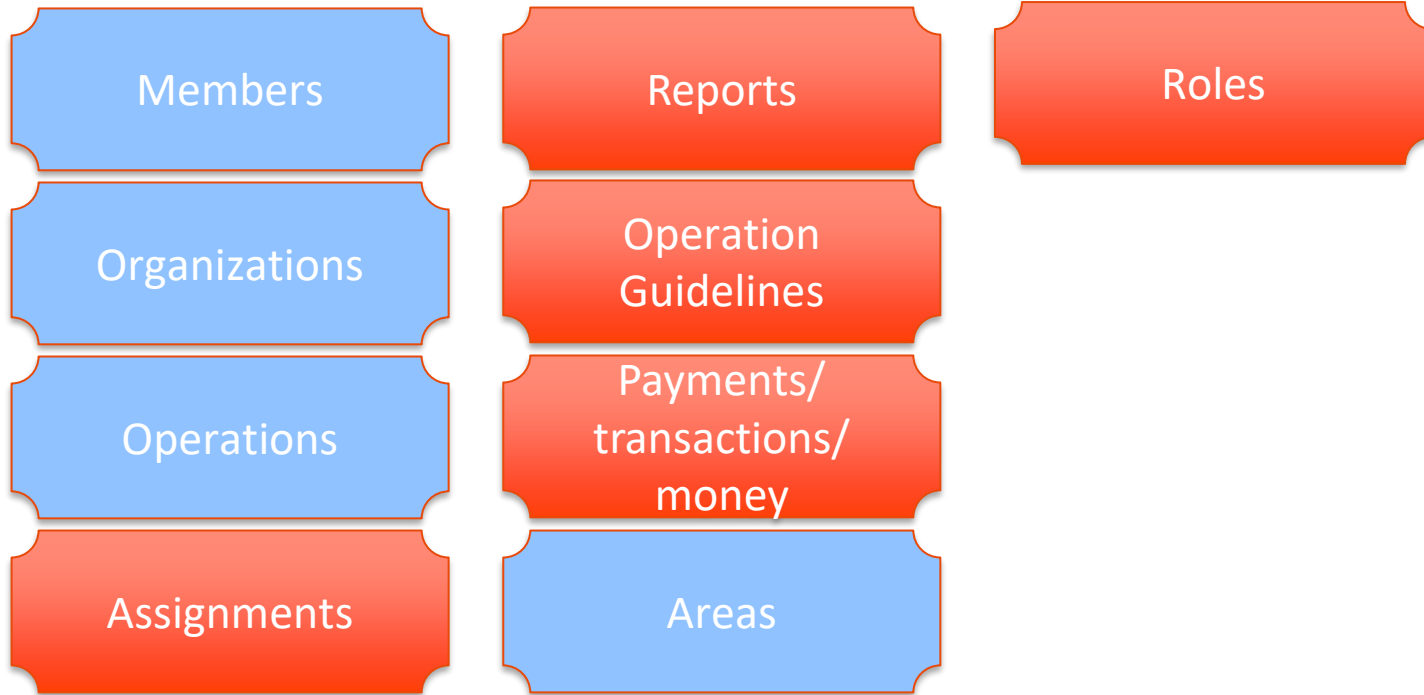
No common definition, but usually we mean:

- It has its own identity
- It exists even if other objects disappear
- The system must manage it explicitly

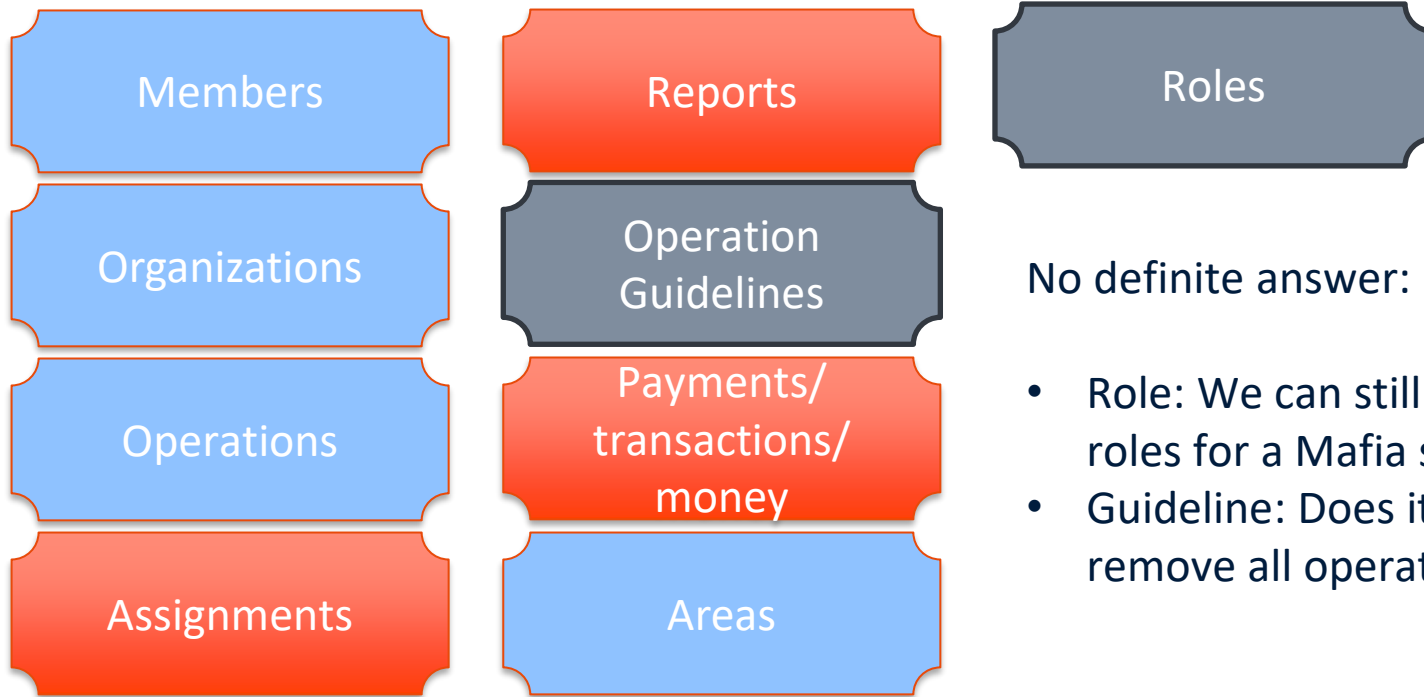
What are the Independent Entities?



Independent Entities



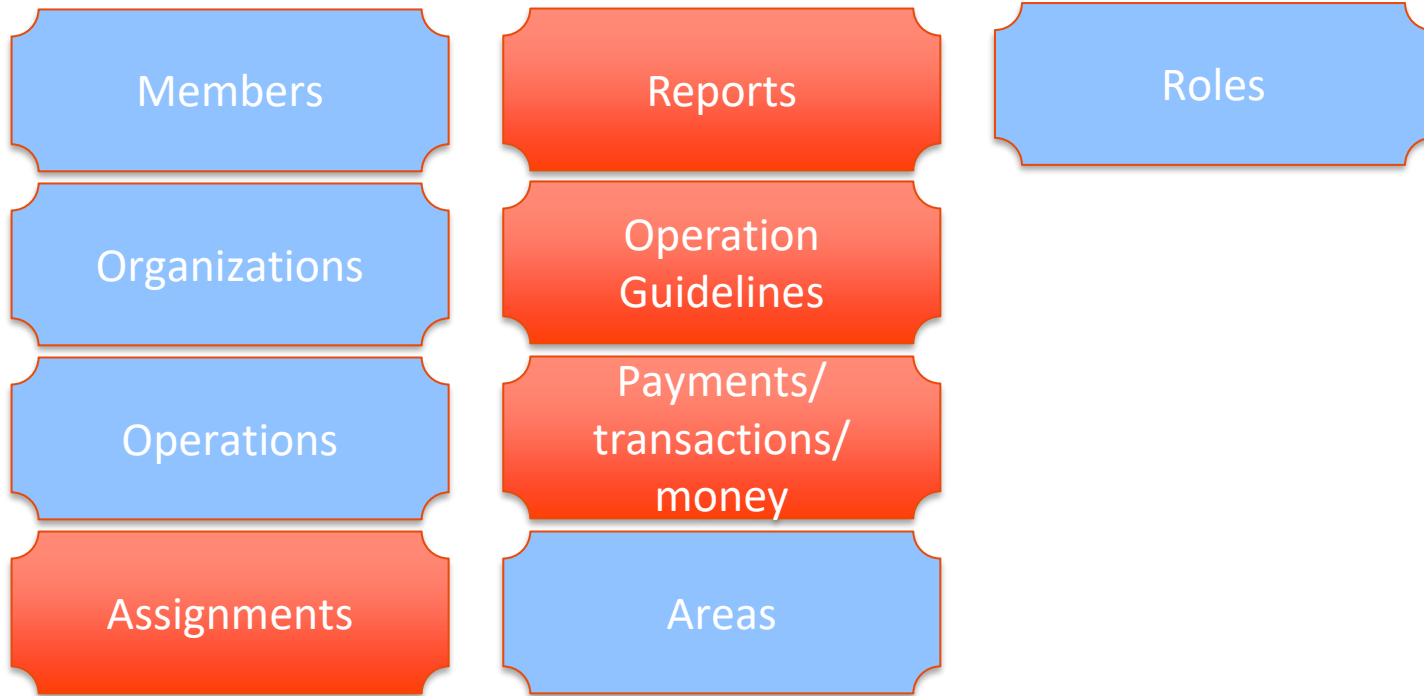
Independent Entities: What about Roles and Guidelines?



No definite answer:

- Role: We can still define stand-alone roles for a Mafia system
- Guideline: Does it still exist if we remove all operations?

Overview



How do we relate this information?

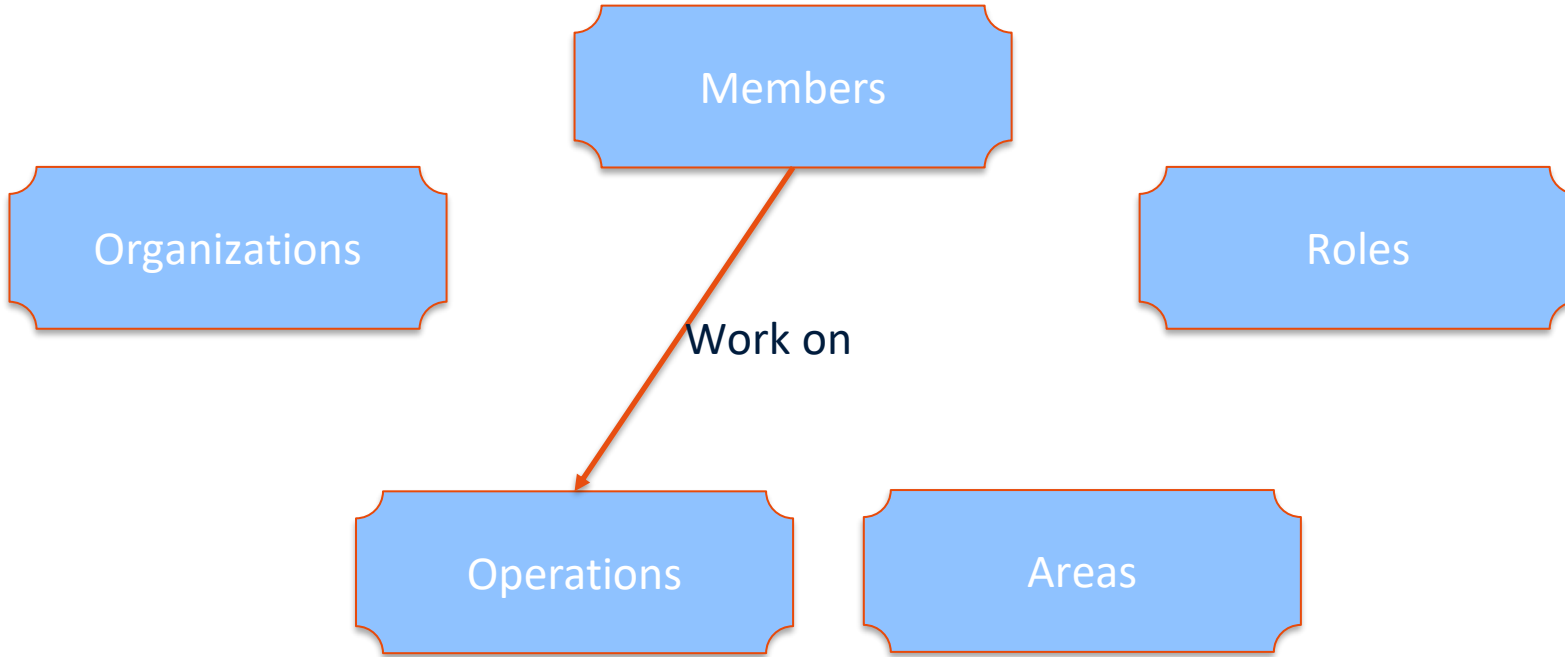


1 min: write down the relationships
(example on next slide)

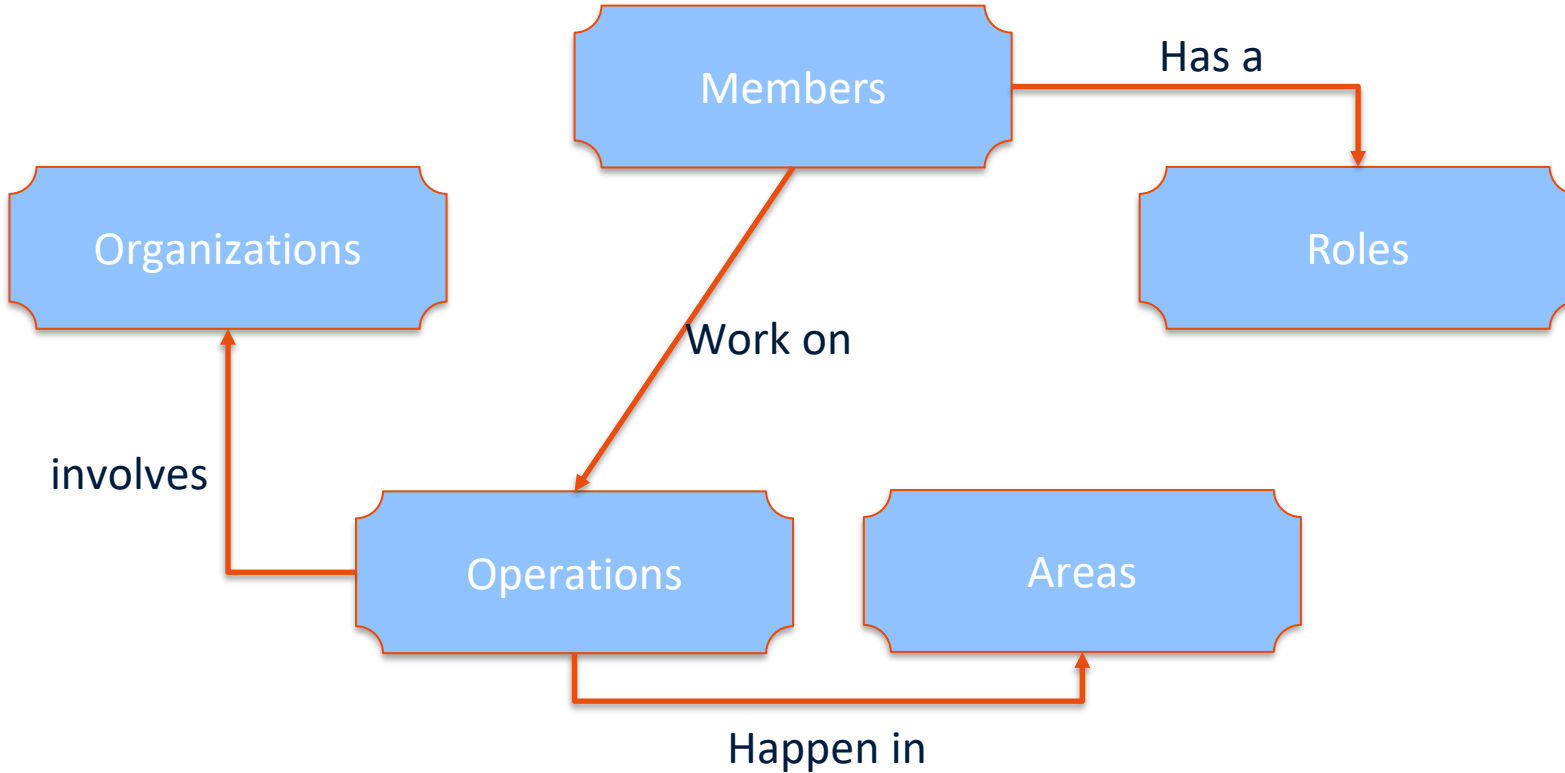
1 min: discuss with your neighbor



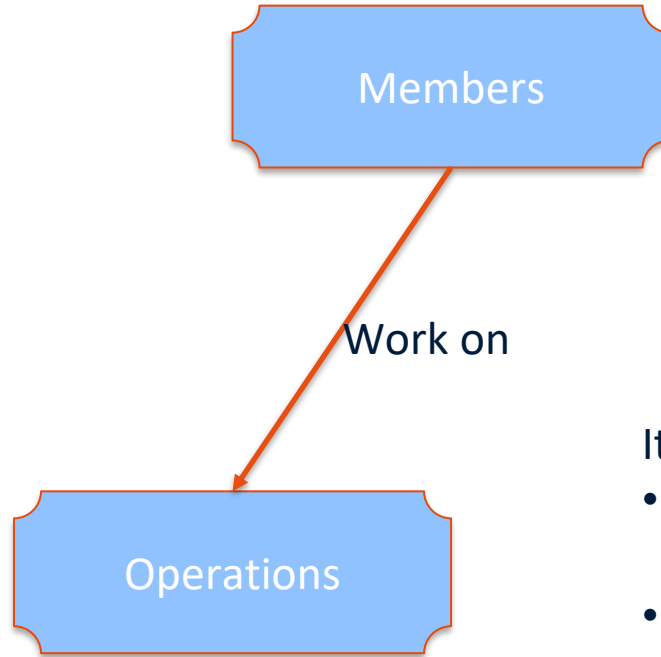
How are these independent objects related?



How are these independent objects related?



How are these independent objects related?

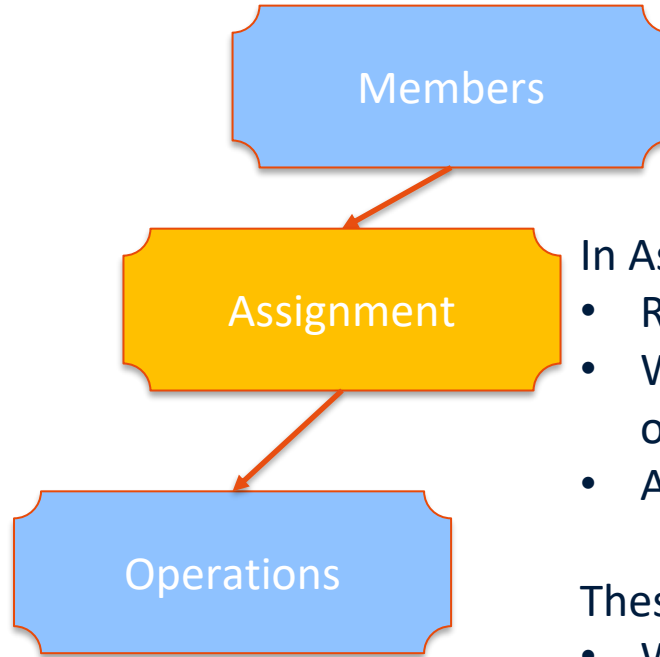


Is it enough to know that a member works on an operation?

It would be nice to know:

- When did a member start working on the operation?
- What is the role of a member within the operation?

Associative Relationship



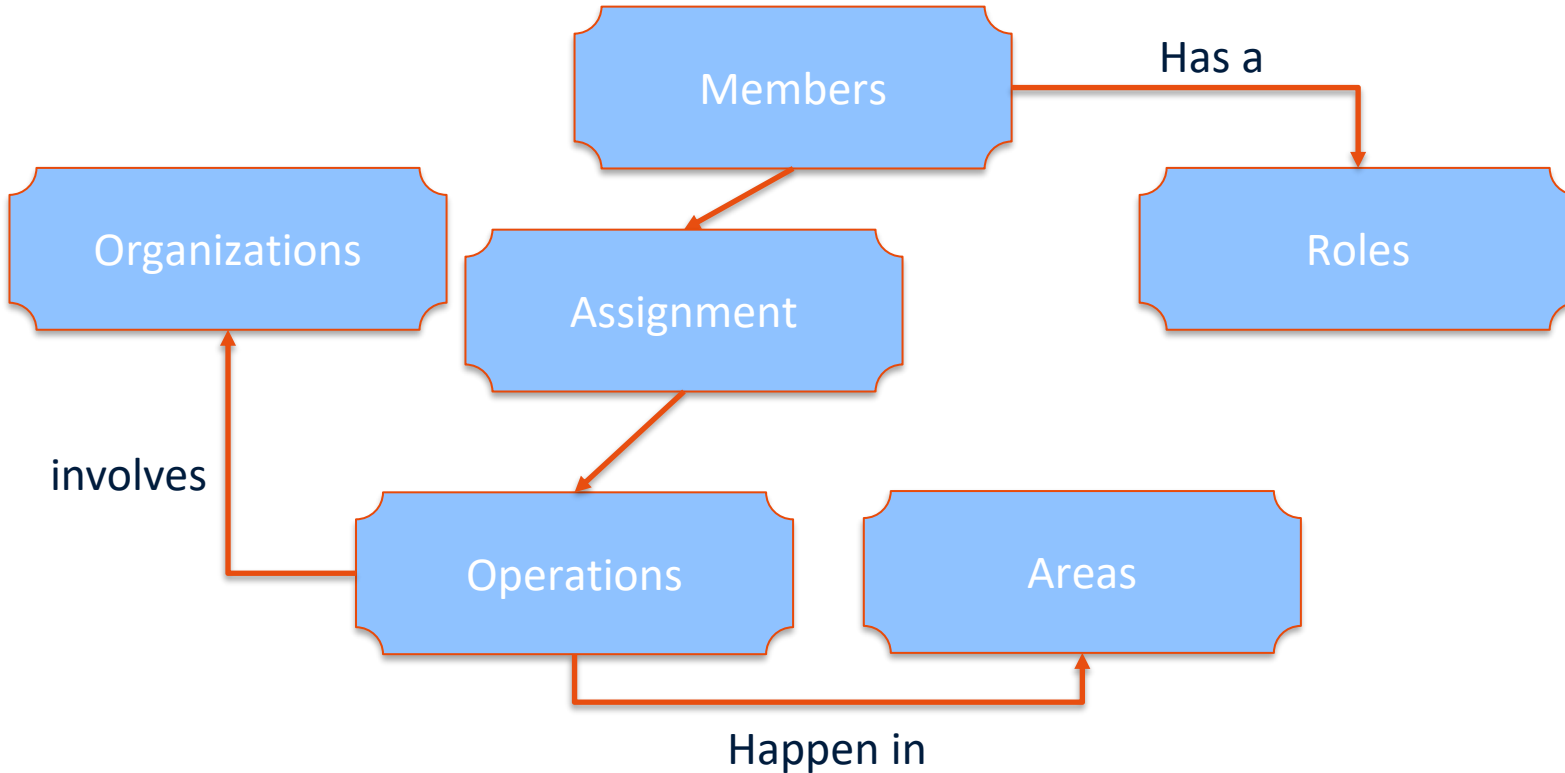
In Assignment, we can store information like:

- Role of member in the operation
- When the member started working on the operation
- Assignment status

These connections are also data

- We can call them relationships

How are these independent objects related?



What about the non-independent entities?

Reports

Payments/
transactions/
money

Operation
Guidelines

Assignments

Event Entities

- Something that happens in time

Normative Entities

- How actors are supposed to behave

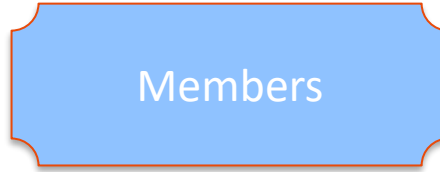
Relationship Entities

- Connection between entities

What attributes does a member have?

For example:

- Alias
- Real name
- Phone
- Status
- Join date



What attributes does an Assignment have?

For example:

- Role in operation
- Start date
- End date
- Status



Please note: Not only the objects can have attributes, but also the relationships

And we can do this for all other entities/relationships

operation
123 operation_id
A-Z operation_name
A-Z operation_type
123 territory_id
🕒 start_date
🕒 end_date
A-Z operation_status
A-Z secrecy_level
A-Z notes

organization
123 organization_id
A-Z organization_name
A-Z organization_type
A-Z region_name
A-Z trust_level
A-Z status
A-Z notes

financial_entry
123 entry_id
123 operation_id
123 member_id
123 organization_id
🕒 entry_date
A-Z entry_type
123 amount
A-Z payment_method
A-Z description
🕒 recorded_at

assignment
123 assignment_id
123 member_id
123 operation_id
A-Z assignment_role
🕒 assigned_on
🕒 released_on
A-Z assignment_status
A-Z notes

role
123 user_id
A-Z role_name

report
123 report_id
123 member_id
123 operation_id
🕒 report_date
A-Z report_category
A-Z summary_text
123 incident_flag
🕒 submitted_at

operation_guideline
123 guideline_id
A-Z operation_type
A-Z guideline_title
A-Z guideline_text
A-Z severity_level
123 active_flag

member
123 member_id
A-Z alias
A-Z first_name
A-Z last_name
A-Z member_role
🕒 join_date
A-Z status
A-Z phone
A-Z notes

area
123 territory_id
A-Z territory_name
A-Z municipality_name
A-Z region_name
A-Z status
A-Z notes

Self-Study – Entity-Relationship Diagrams

- You know already:
 - What is an entity and that different types exist
 - What are relationships
 - What are attributes of entities/relationships
- What is missing:
 - Type of relationship (e.g. many-to-many, one-to-many)
 - ER diagram notation
 - Applications that help you draw the ERDs
- Self-Study on ERDs:
 - <https://www.youtube.com/watch?v=QpdhBUYk7Kk>
 - <https://www.youtube.com/watch?v=-CuY5ADwn24>

What did we do?

- Step 1: Identify information we need to store
- Step 2: Decide about types of entities
- Step 3: Identify relationships
- Step 4: Refine attributes

Running Example

- We just created our running example for the course together
- Will be used throughout the course to illustrate concepts and perform exercises on

Short Break

- **10 minutes**



How can we store the data?



Can we use a file based system?

- Store information in files:
 - members.txt,
 - organisations.txt,
 - operations.txt,
 - money.txt,
 - daily-log.txt
- Data access:
 - Write a program that opens one or more files, scanning through them and reading/writing information in them 

Problems in a file-based system

- Access Control 
 - Fine-grained permissions
 - Accountants see `money.txt` only (no names and addresses of our soldiers) to prevent... ambitious ideas (takeovers)   
- Access patterns and performance
 - How to identify shops overdue on payments for over a month.  
 - Scan huge file, sort by time, pick the oldest, measure time?

Problems in a file-based system

- Atomicity 
 - Ensure money transfers complete fully or not at all
 - *"Money vanishes only if you plan to disappear too."*
- Consistency 
 - Maintain data in a valid state at all times.
 - E.g. no two operations share a name.

Problems in a file-based system

- Isolation  
 - Soldiers add to `daily-log.txt` simultaneously.
 - Prevent overwrites to avoid firing for low productivity
- Durability  
 - In case of a computer crash, ensure data persists and remains accurate after crashes
 - Payments to cops must be reliably recorded (avoid double payments).

Limitations of a file-based system

- Separation and Isolation of Data
 - Data is stored in different files
 - Makes cross-referencing or combining data challenging
- Duplication of Data
 - Wasteful, costs time and money
 - Leads to loss of data integrity (inconsistent data)
- Data Dependence
 - Tight connection between data and the programs that process it
 - Makes changes to the files structure or format challenging
- Incompatible file formats
 - Structure of files is embedded in the application programs -> structures are dependent on application programming language
 - Creates the need of converting files to common format (time-consuming)
- And many others

Summary: What are the requirements for our Mafia information system?

- Efficient Data Access
- Access Control
- Atomicity
- Consistency
- Isolation
- Durability

THE SOLUTION

- Thankfully, there is something called **Database Systems**.
- These are designed to overcome these limitations

Database Systems

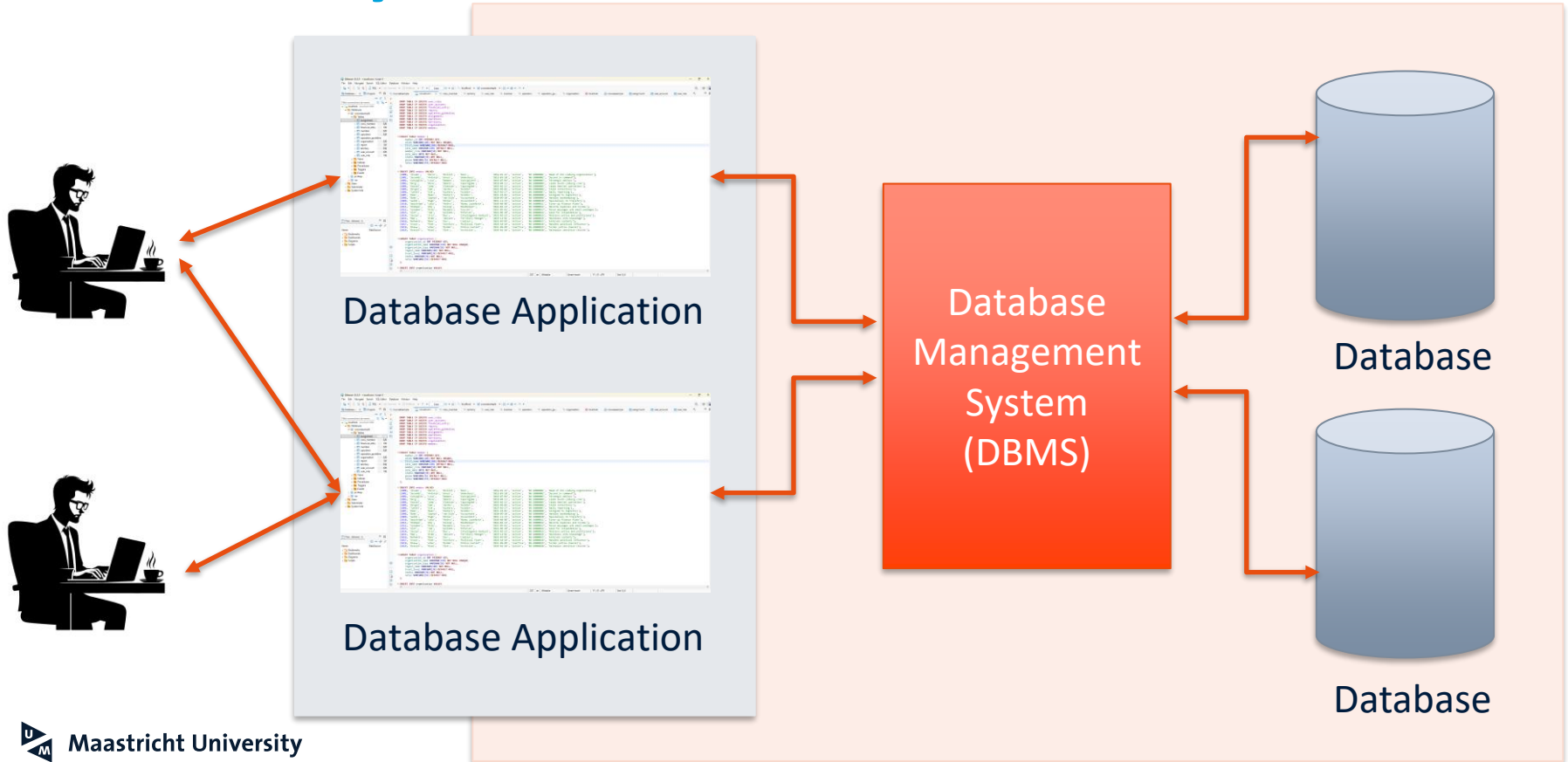


Database Systems - Definitions

- Database
 - A collection of related data
- Database Management System
 - The software that manages and controls access to the database
- Database Application
 - A program that interacts with the database at some point in its execution

Database System

Database: A collection of related data



Database Systems - Definitions

- Database
 - A collection of related data
- Database Management System
 - The software that manages and controls access to the database
- Database Application
 - A program that interacts with the database at some point in its execution

Database Definition (1)

Connelly and Begg:

- A shared collection of **logically related** data and its description, designed to meet the informational needs of an organization.

Emphasize:

- Logically related data: no random collection of data
- Organizational purpose
- Metadata (data description)
- Requirements-driven design

Database Definition (2)

Ullman and Widom:

- Term database refers to a **collection** of data that
 - Is **exists over a long period of time**
 - and **managed** by a data base management system (DBMS)

Emphasize:

- A database is persistent organisational memory
- data under systematic management

Database Systems - Definitions

- Database
 - A collection of related data
- Database Management System
 - The software that manages and controls access to the database
- Database Application
 - A program that interacts with the database at some point in its execution

What is a DBMS?

- Definition (Connelly and Begg)
 - A software system that enables users to **define**, **create**, **maintain** and **control access** to the database

What is a DBMS

- Allows users to **define the logical structure** of a database through a data-definition language (DDL)
- Gives users the ability to **manipulate and retrieve data** from the database through a data-manipulation language (DML)
- Support the storage of very large amounts of data over a **long period of time**, allowing **efficient access** to the data
- Enable **durability**, the recovery of the database in the face of failures
- Control **shared access** to data from many users at once

Example: Online Bookseller

- Business Model
 - Buy books, store them, and sell via a website.
- Essential Data
 1. Books 
 2. Customers 
 3. Orders 
 4. Preferences 

Example: Online Bookseller

- Data operations
 - Insert 
 - Remove 
 - Search by author/title 
 - Change status (stock) 

What does the DBMS do in the Bookseller Example?

-  Describe real-world entities in the database (e.g., Books) using the Data Definition Language (DDL)
-  Read and Write to this database (e.g. add a sale and reduce the stock) using the Data Manipulation Language (DML)
-  Stores data over a long period of time (users may ask for a years-old receipt)

What does the DBMS do in the Bookseller Example?

-  Ensures Durability (What if a user places an order and is paying for the book but our website stops due to a crash? What happens to the order in the database?)
-  Ensures Concurrency (What if four users are trying to buy Darth Vader Autobiography at the same time and we only have one copy?)

Examples of DBMS

Large enterprise
relational systems

Oracle, IBM Db2,
Microsoft SQL Server

Used in banks, governments,
large corporations

Open-source relational
systems

PostgreSQL, MySQL,
SQLite

Research, Education, ...

In-memory/high-
performance systems

SAP HANA

Designed for real-time
analytics on massive data

Non-relational (NoSQL)
systems

MongoDB, Neo4j,
HBase

Schema is flexible, massive
distribution storage, different
querying languages (speed)

Database Systems - Definitions

- Database
 - A collection of related data
- Database Management System
 - The software that manages and controls access to the database
- Database Application
 - A program that interacts with the database at some point in its execution

(Database) Application Programs

- A computer program that interacts with the database by issuing an appropriate request (typically an SQL statement) to the DBMS

Where do we use Database Systems?

- Buying groceries from Supermarket
 - Product Inventory
 - Point of Sale System
 - Customer/rewards database
 - ...
- Spotify and Netflix Subscription
 - User accounts & authentication database
 - Content Catalog Database
 - Recommendation system (ML database)
 - ...
- Buying stuff using your credit card
 - Check whether you have enough credit left to make the buying
 - Credit card on the list of stolen cards?
 - ...
- Booking a vacation with travel agent
- Using the local library
- More can be found in Connolly and Begg

Why are we still teaching Relational Databases?

- They still run the world
- They teach Conceptual Data Modelling
- For your career (most job postings mention SQL)
- SQL is one of the most standardised languages in computer and data science and is easily teachable

Questions?

Discussion board on Canvas

Attend tutorials 😊

Personal issue via mail:
denise.slenter@maastrichtuniversity.nl



See you tomorrow

